

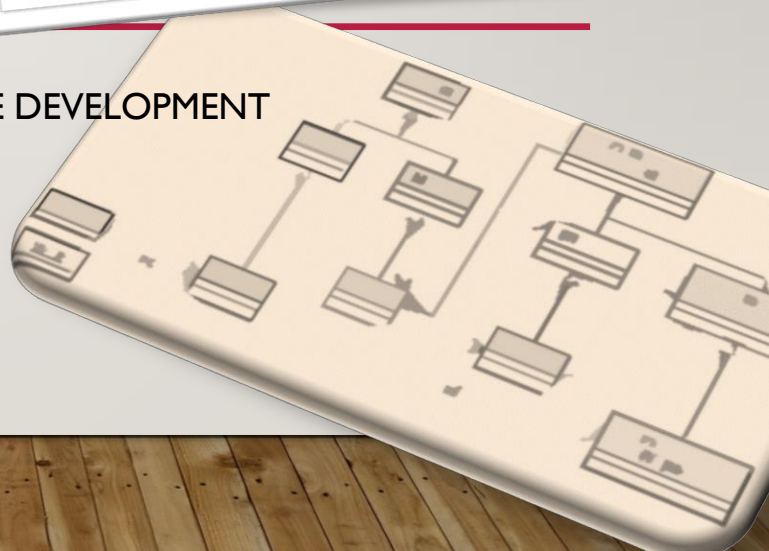
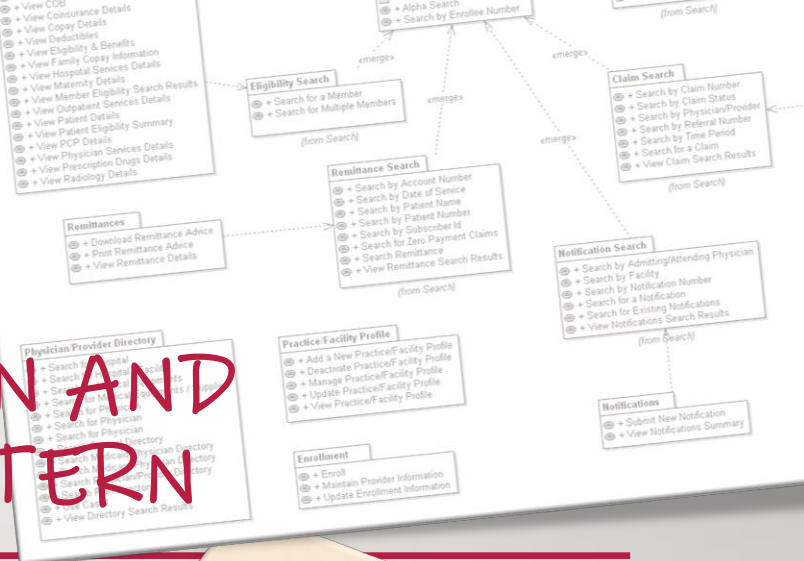


THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

DESIGN PATTERNS - INTRODUCTION AND COMPOSITE PATTERN

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH



AGENDA

- Introduction to Design Patterns
- Pattern of the day
 - Composite Pattern

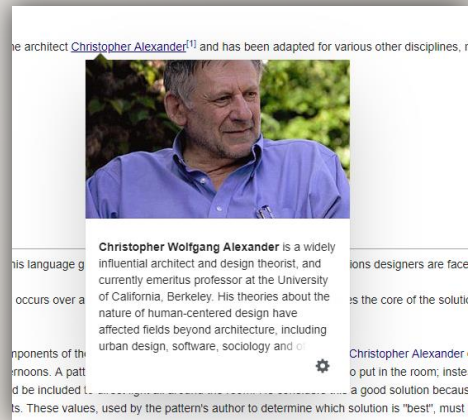
PREVIOUS LECTURE

- *OBJECTIVES AND PRINCIPLES FOR SOFTWARE DESIGN*

- **Robustness**, **Flexibility**, Correctness, Reusability, Efficiency, Testability
- SOLID Principles
 - Single Responsibility Principle (SRP)
 - Open Closed Principle (OCP)
 - Liskov Substitution Principle (LSP)
 - Interface Segregation Principle (ISP)
 - Dependency Inversion Principle (DIP)

DESIGN PATTERNS

- A **design pattern** captures a **proven** solution to a **common** design problem in a **specific** context.
- A **design pattern** is a:
 - **reusable solution** encountered in a particular context
 - **formally documented** for software developers confronting those general design problems
 - **enables learning from the experience** of other software developers
 - solution that has been **proven as an effective method** of solving a problem in a given context.

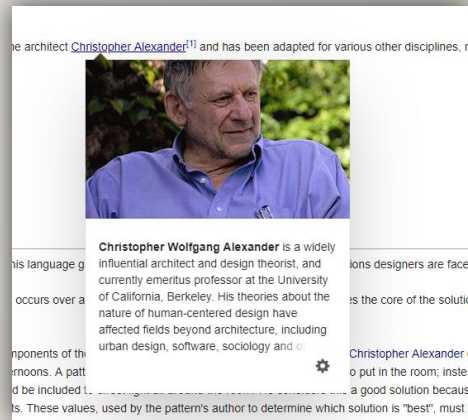


Alexander, Christopher (1977). *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press.

DESIGN PATTERNS

“The elements of this language are entities called patterns. Each pattern describes a problem that occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”

— Christopher Alexander



Alexander, Christopher (1977). *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press.

DESIGN PATTERNS

- Common **design problems** include:
 - How to ensure that a particular class has only one instance which has a global point of access?
 - How to define the skeleton of an algorithm in a superclass and allow subclasses to redefine certain steps without changing the algorithm's structure?
 - How to compose objects into tree structures, represent *whole-part* hierarchies, where leaf and composite nodes are treated uniformly?
 - How to make objects with incompatible interfaces work together?
 - How to deal with the situation where you know that you need to create an object – but you do not know which particular class of object to instantiate?

DESIGN PATTERNS

- Common **design problems** include:
 - How to ensure that a particular class has only one instance which has a global point of access (**Singleton** pattern)
 - How to define the skeleton of an algorithm in a superclass and allow subclasses to redefine certain steps without changing the algorithm's structure (**Template Method** pattern)
 - How to compose objects into tree structures, represent *whole-part* hierarchies, where leaf and composite nodes are treated uniformly (**Composite** pattern)
 - How to make objects with incompatible interfaces work together (**Adapter** pattern)
 - How to deal with the situation where you know that you need to create an object – but you do not know which particular class of object to instantiate (**Factory** pattern)

THE 'GANG OF FOUR'

Gang of Four (GoF)



■ Ralph Johnson, Richard Helm, Erich Gamma, and John Vlissides (left to right)

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.E. Eicher / Corbis Art - Baum - Holland. All rights reserved.

Foreword by Grady Booch

ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES



PATTERN DESCRIPTION

Context:

- The general situation in which the pattern applies

Problem:

- A short description of the design problem at hand

Forces:

- The constraints to be kept in mind during problem solution

Solution:

- The recommended problem solution in the given context.

Antipatterns:

- Wrong solutions – not suited to current context.

Related patterns:

- Other patterns to which this pattern relates (*uses, is used by, is similar to, etc.*)

References:

- Who developed or inspired the pattern.

GENERAL HIERARCHY PATTERN

- **Context:**
 - Objects in a hierarchy can have one or more objects above them (superiors),
 - and one or more objects below them (subordinates).
 - Some objects cannot have any subordinates
- **Problem:**
 - How do you represent a hierarchy of objects, in which some objects cannot have subordinates?
- **Forces:**
 - You want a flexible way of representing the hierarchy
 - that prevents certain objects from having subordinates
 - All the objects have many common properties and operations
- **Reference:**
 - E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, October 1994.

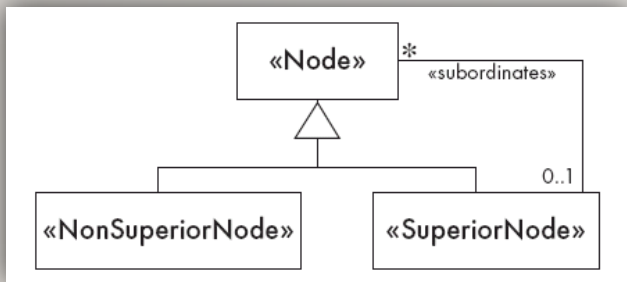
GENERAL HIERARCHY PATTERN

Examples of design problems

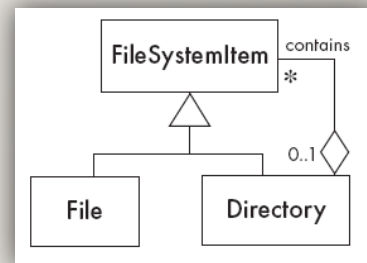
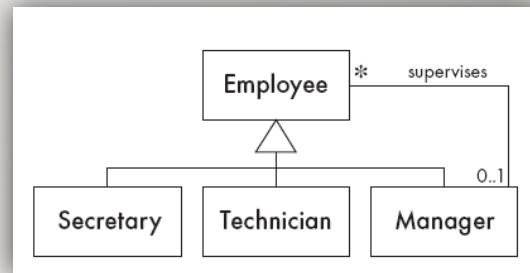
- A **file-store** is organised into directories and files. A file doesn't contain other files or directories. A directory is a special kind of file that may contain other files (including directories).
- A **nesting shape** is a special kind of shape which may contain other (nested and non-nested) shapes. The contained shapes move and bounce around within the nested shape.
- **Employee hierarchies** allow for some employees to have supervisory roles while others don't play the supervisor role.

GENERAL HIERARCHY PATTERN

- Solution (general):***



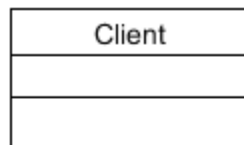
- Examples:***



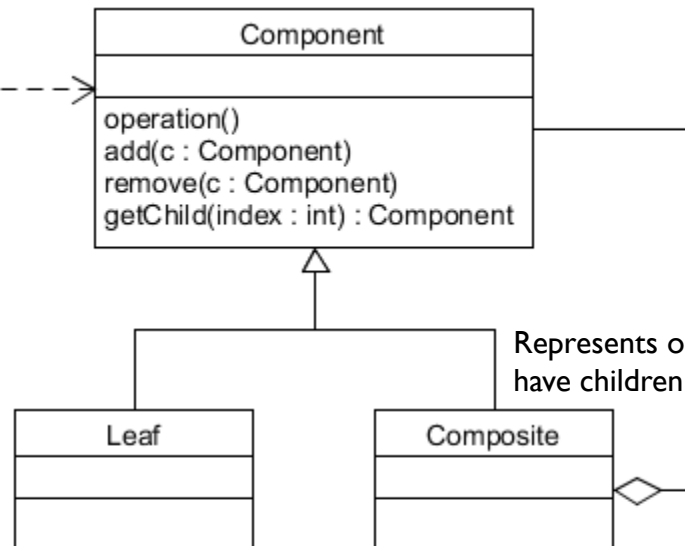
COMPOSITE PATTERN

Defines an “interface” for objects in the composition; implements default behaviour; defines an interface for managing children

Manipulates objects using the Component “interface”



- compose objects into tree structures to represent whole-part hierarchies.
- composite lets clients treat individual objects and compositions of objects uniformly



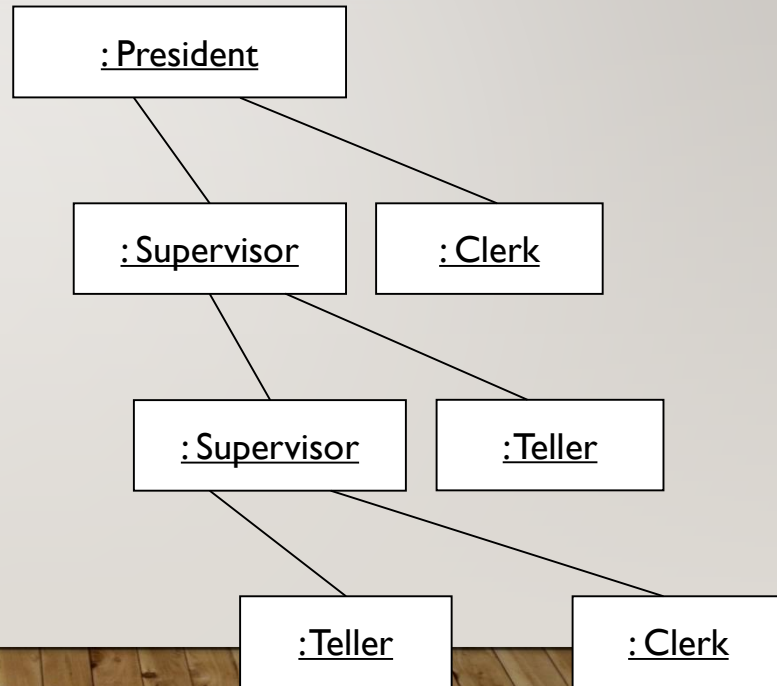
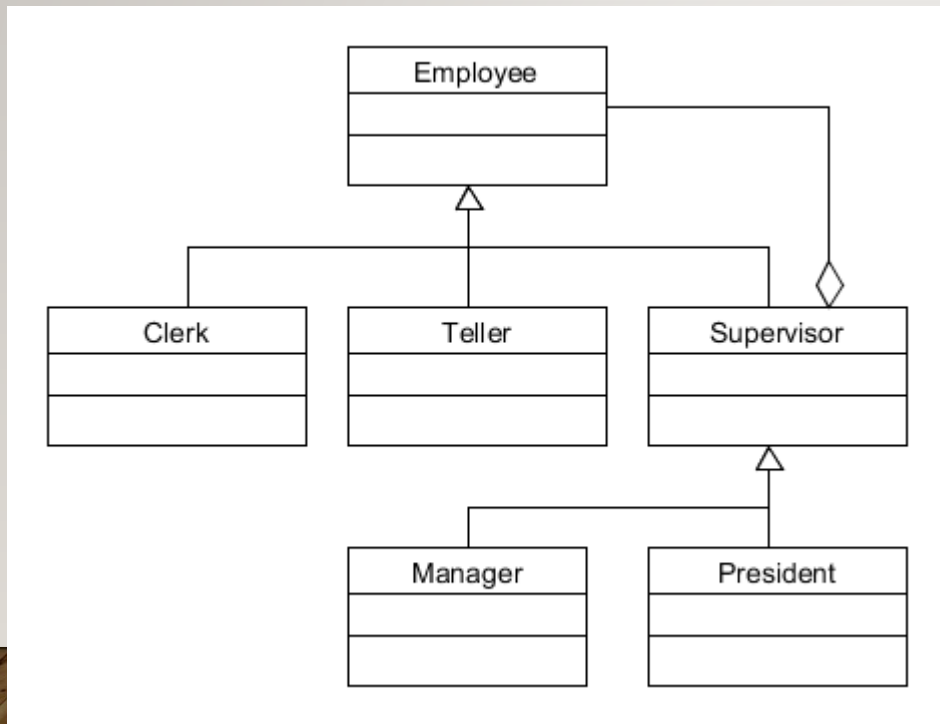
Represents objects that can have children

Represents objects that have no children; implements behaviour for such objects

```

public void
operation() {
    for all c in children
        c.operation();
}
    
```

APPLICATION #1: EMPLOYEE ORGANISATION



APPLICATION #2: FILE-STORE

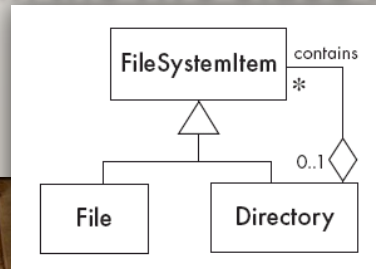
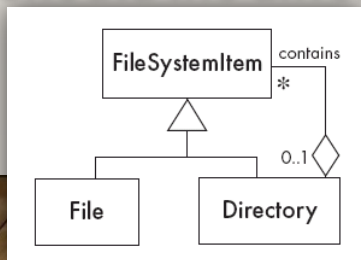
```
class FileSystemItem {
    ...
}

class Directory extends FileSystemItem {
    List<FileSystemItem> contents;
    ...
}
```

```
class FileSystemItem {
    Directory parent;
    ...
}

class Directory extends FileSystemItem {
    List<FileSystemItem> contents;
    ...
}
```

How should the difference in code reflect in the two class diagrams?



CHILD MANAGEMENT AND ITERATION

- **Parent references**
 - Maintaining bidirectional links between children and their parents is often desirable
 - It's essential to maintain the invariant that all children of a composite have as their parent the composite that in turn has them as children
- **Child ordering**
 - In many cases, the ordering of children within a Composite is important
 - Child management operations should be implemented with this ordering in mind
- **Caching**
 - Composite can cache traversal information about its children
 - Changes to children may invalidate the cache however
 - Children should maintain a reference to their parent so that they can inform the parent of the invalidated cache

CONSEQUENCES

- **Composite pattern**
 - enables **recursive structures** to be implemented
 - Where a client expects a primitive, it can also take a composite
 - **simplifies clients** since they work with the Component interface
 - Client code does not need conditional statements to check the type of object it is working with
 - makes it **easier to add new components**
 - New subclasses of Composite or Leaf can be added without needing to change existing code
 - can **make the design overly general**
 - The disadvantage of making it easy to add new components is that it makes it harder to restrict the components of a composite
 - Run-time checking may be necessary

SUGGESTED TASKS

- Implement classes File and Directory and **build a small composite structure** of File/Directory objects
- Which **design objectives** are helped by the **design pattern** studied in today's class?
- Can you see **some instances of SOLID principles** being followed (or not followed) – in the examples of this design pattern?

BENEFITS OF DESIGN PATTERNS

- Design patterns offer several benefits:
 - They are known to work because they are **derived from practical experience**
 - They are **sufficiently abstract** to be applicable to a generic class of problem
 - They provide a **common vocabulary** which enhances communication between developers
 - They contribute to **system documentation**
 - They **assist developers in maintaining software** since they prescribe rules that should not be broken

REVIEW

- **Learnt something about:**
 - **Design patterns**
 - Design patterns are proven, generalisable and reusable design techniques for common design problems
 - Design patterns have stated benefits and are widely used in modern object-oriented software development
 - **Composite Pattern**
 - Recursive and partitioning design pattern
 - describes a group of objects that are treated the same way as a single instance of the same type of object
 - The intent of a composite is to "compose" objects into tree structures to represent part-whole hierarchies
 - Implementing the composite pattern lets clients treat individual objects and compositions uniformly.

REFERENCES

➤ [Books]

- Chapter 6 – Design Patterns (**Canvas**), Object-Oriented Software Engineering: Practical Software Development using UML and Java by Timothy Lethbridge and Robert Laganieri
- Design Patterns: Elements of Reusable Object-Oriented Software by John Vlissides, Ralph Johnson, Richard Helm, Erich Gamma
- Agile Software Development, Principles, Patterns, and Practices by Robert C Martin

➤ [Media]

- <http://www.javaworld.com/javaworld/jw-10-2001/jw-1012-designpatterns.html>
 - A JavaWorld introductory article on design patterns

➤ [See more on Canvas Lecture page]